Coding & Solution

Date	25 September 2026
Team ID	SWTID-2026-3976
Project Name	AI FAQ Assistant API (AnswerBot)
Maximum Marks	5 Marks

Coding & Solution

The AI FAQ Assistant API implements the core problem statement: a secure backend to create, search and AI-generate FAQs. Full source, configuration and setup steps are provided with the project.

Solution Summary

Field	Details
Repository Link / URL	**[Add GitHub repository URL here before submission]**
Programming Language(s)	JavaScript (Node.js 18+)
Framework(s) Used	Express.js, Mongoose ODM
Key Features Implemented	User registration and login (JWT, bcrypt); protected profile; FAQ create, list, view, update, delete; keyword search; category enum; AI answer and AI FAQ generation with Google Gemini; validation and centralised error handling.
Pending / Incomplete Features	Admin/editor role management screens, FAQ likes and answer feedback, tags, rate limiting and usage analytics (planned for future phases).
Setup / Run Instructions	1) Install Node.js 18+ and MongoDB access. 2) Run `npm install` (express, mongoose, dotenv, cors, jsonwebtoken, bcrypt, @google/genai). 3) Create a `.env` file holding the server port, MongoDB connection string, JWT secret and Gemini API key. 4) Start the server with `npm run dev` (or `npm start`); it listens on port 5001. 5) Test the endpoints with the Postman collection.

API Endpoints

Method	Route	Access	Purpose
POST	/api/auth/register	Public	Register a user and return a JWT
POST	/api/auth/login	Public	Log in and return a JWT
GET	/api/auth/profile	Private	Get the logged-in user's profile
GET	/api/faqs	Public	List all FAQs
GET	/api/faqs/search?q=	Public	Keyword search
GET	/api/faqs/:id	Public	Get one FAQ
POST	/api/faqs	Private	Create an FAQ
PUT	/api/faqs/:id	Private	Update own FAQ
DELETE	/api/faqs/:id	Private	Delete own FAQ
POST	/api/ai/answer	Private	AI answer to a question

Method	Route	Access	Purpose
POST	/api/ai/generate-faq	Private	AI-generated FAQ from a topic

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes (Git / GitHub)

Additional Notes / Comments: Secrets (JWT secret, Gemini key, database URI) are read from `.env` and must never be committed to the repository.